

GrpcCall Operator 设计文档

背景和需求

在现有架构中，`unit-operator` 用于管理有状态服务（如 MySQL、PostgreSQL、Redis、Kafka、Elasticsearch 等）的生命周期。每个 `unit` 所生成的 Pod 中，包含一个主容器（通常以 `supervisord` 作为进程入口）和一个名为 `unit-agent` 的 sidecar 容器。`unit-agent` 通过 `gRPC` 接口对服务进程进行控制（例如配置渲染、进程生命周期管理、动态参数注入、物理备份和还原等）。外部控制系统不能直接访问集群内部的 `unit-agent`，因此我们最初的设计是通过创建 Job 的方式来完成一次性任务。

在旧架构中：

- 上层管理系统（如 `UPM-API-Server`）调用 Kubernetes API 创建一个 Job。
- Job 使用封装了 `grpcurl` 和 `toolkit` 脚本的镜像。
- 每次执行任务都需要启动一个新的 Pod，运行命令如 `toolkit mysql backup`。
- 通过环境变量传递参数（如 `BACKUP_MODE`、`S3_BUCKET` 等）。
- 执行结果需通过查看 Job Pod 的日志才能获取，流程繁琐且资源开销较大。

为优化这一流程，我们设计了新的方案：

- 引入一个名为 `GrpcCall` 的自定义资源（CRD），代表一次 `unit-agent` 的调用。
- 上层系统只需创建一个简单的 `GrpcCall` 对象，即可触发对应的调用逻辑。
- `GrpcCall` Operator 监听 CR 的创建事件，并在内部直接构造 `gRPC` 客户端，向目标 `unit-agent` 发送请求。
- 无需额外创建 Pod，大大简化了流程并减少资源开销。
- `GrpcCall` 的 `status` 字段可以结构化记录调用结果（如 `Success`、`Failed`、`Timeout` 和 `message`），便于诊断和展示。

通过这一新架构，系统调用 `unit-agent` 的能力更轻量、直接且可观测性更强，避免了使用 Job 的冗余开销和复杂的排查路径，提升了平台操作的效率和用户体验。

GrpcCall CRD 设计

我们定义一个新的 CRD：`GrpcCall`。其 `spec` 包含以下字段：

- `targetUnit`（字符串）：目标 Unit 的名称，用于定位对应的 Pod。
- `type`（字符串）：服务类型，例如 `mysql` 或 `postgresql`。
- `action`（字符串）：操作名称，例如 `Backup`、`Restore`、`ReloadConfig` 等。

- `parameters`（键值对 Map）：额外参数列表，可用于传递具体操作所需的选项（示例：`{"mode": "full", "bucket": "s3://mybucket"}`）。

`status` 字段用于记录调用结果：

- `status.result`（枚举）：调用结果，可取 `Success`、`Failed` 或 `Timeout`。
- `status.message`（字符串）：操作结果的详细信息或错误消息。

示例的 GrpcCall 实例（YAML）如下：

```
yaml
1  apiVersion: upm.syntroupycloud.io/v1alpha1
2  kind: GrpcCall
3  metadata:
4    name: mydb-backup
5  spec:
6    targetUnit: mydb-instance
7    type: mysql
8    action: backup
9    parameters:
10     mode: full
11     bucket: s3://mybucket
```

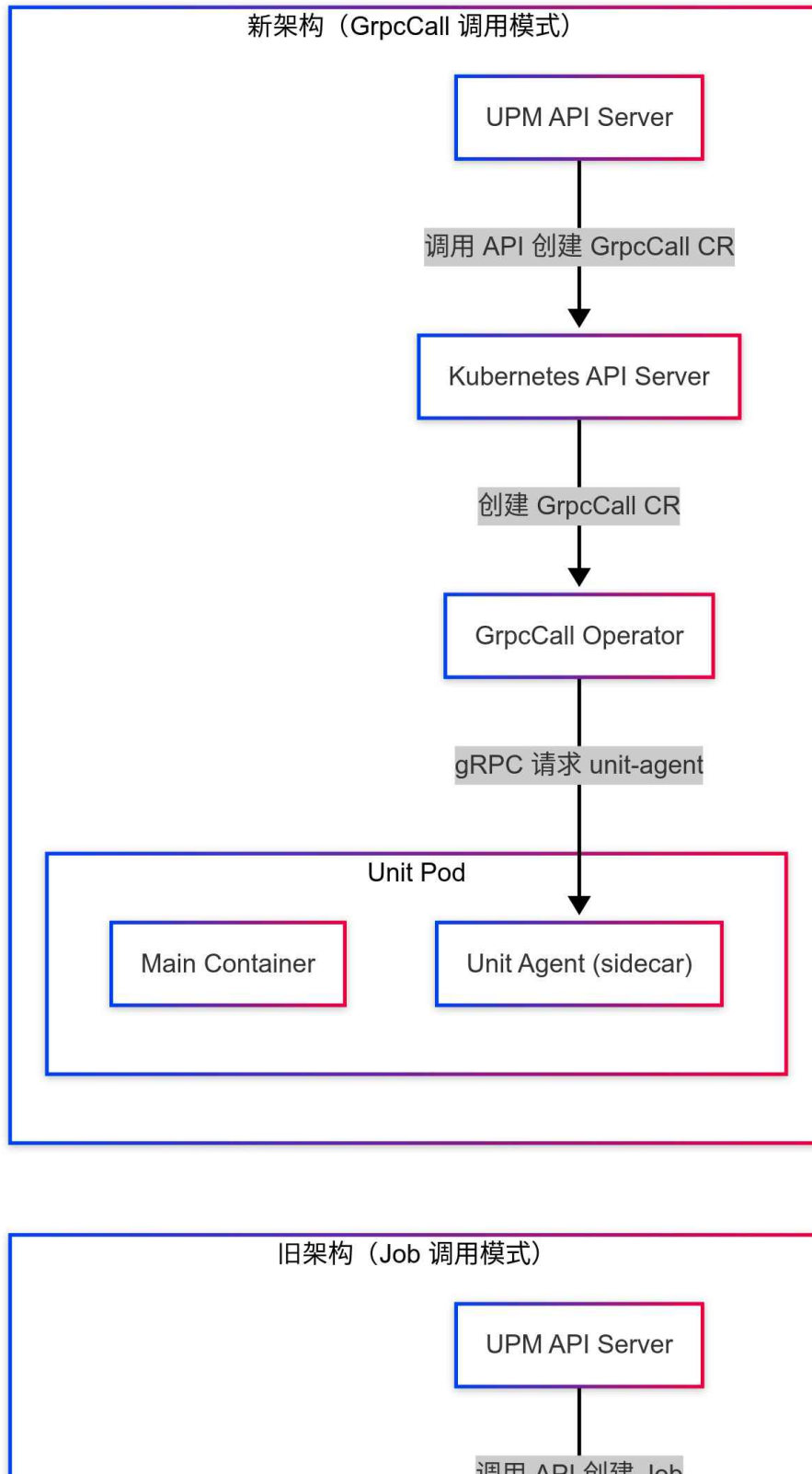
架构图

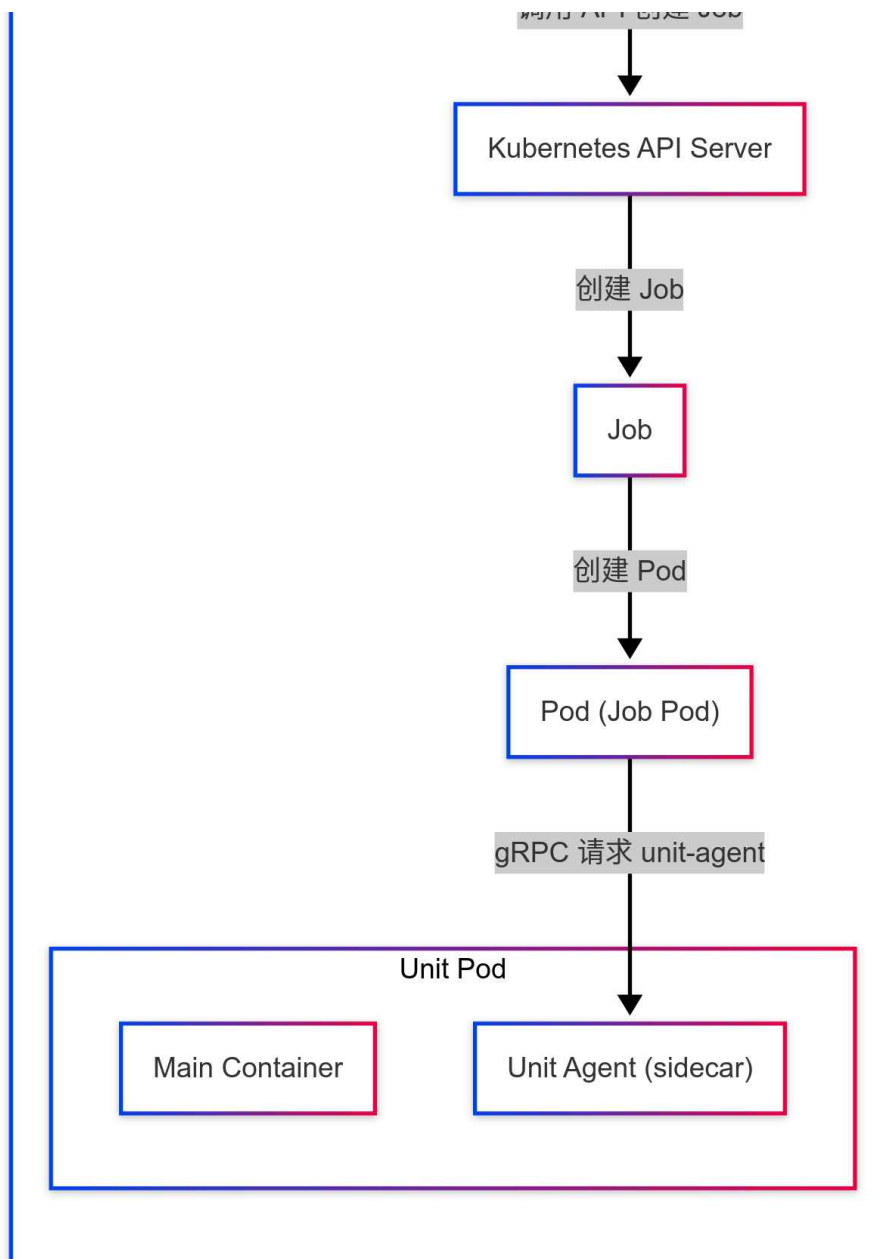
```
mermaid
1  graph LR
2    subgraph "旧架构（Job 调用模式）"
3      UPM_old["UPM API Server"]
4      K8s_old["Kubernetes API Server"]
5      Job_obj["Job"]
6      JobPod["Pod（Job Pod）"]
7      subgraph UnitPodOld [Unit Pod]
8        Unit_old["Main Container"]
9        Unit_agent_old["Unit Agent (sidecar)"]
10      end
11      UPM_old -- "调用 API 创建 Job" --> K8s_old
12      K8s_old -- "创建 Job" --> Job_obj
13      Job_obj -- "创建 Pod" --> JobPod
14      JobPod -- "gRPC 请求 unit-agent" --> Unit_agent_old
15    end
16
17  subgraph "新架构（GrpcCall 调用模式）"
18    UPM_new["UPM API Server"]
19    K8s_new["Kubernetes API Server"]
```

```

20 GrpcOperator["GrpcCall Operator"]
21 subgraph UnitPodNew [Unit Pod]
22     Unit_new["Main Container"]
23     Unit_agent_new["Unit Agent (sidecar)"]
24 end
25 UPM_new -- "调用 API 创建 GrpcCall CR" --> K8s_new
26 K8s_new -- "创建 GrpcCall CR" --> GrpcOperator
27 GrpcOperator -- "gRPC 请求 unit-agent" --> Unit_agent_new
28 end

```





Operator 逻辑

GrpcCall Operator 的核心逻辑如下：

1. 监听 GrpcCall 资源：Operator 启动后注册 GrpcCall CRD，并监听新增或更新的 GrpcCall 实例。
2. 获取目标 Pod：根据 `targetUnit` 查找对应的 Pod。例如，可使用与 Unit 关联的标签或命名约定来定位 Pod。
3. 建立 gRPC 连接并调用接口：Operator 直接对目标 Pod 的 unit-agent 容器 IP 和端口发起 gRPC 调用（例如 `unit-agent:9090`），执行指定的 `action`。
4. 传递参数并处理响应：将 `parameters` 中的键值作为请求参数传入，等待 unit-agent 返回结果或错误。可通过 `context.WithTimeout` 实现调用超时。
5. 更新 CRD 状态：根据 gRPC 调用结果，更新 GrpcCall 对象的 `status.result` 和 `status.message` 字段。例如，调用成功则 `Success` 并附上返回信息；发生错误则 `Failed` 并附上错误原因；超时则标记 `Timeout`。

整个过程只执行一次，不做自动重试或批量处理。在调用完成后，GrpcCall 资源可以由用户或自动化工具删除。Operator 不保持连接状态，每个 CR 独立触发一次调用。控制器的工作流程遵循 Kubernetes 的控制循环模型，即将 spec 中描述的期望操作与 status 中反映的实际执行情况进行协调。

状态字段定义

在 GrpcCall CRD 的定义中，`status.result` 使用枚举类型（如 `Success|Failed|Timeout`）来反映调用结果，`status.message` 用于提供调试或错误信息。例如：

```
yaml
1  status:
2    result: Success
3    message: "Backup completed, files written to s3://mybucket/..."
```

若遇超时或调用异常，则可设置 `result: Failed`（并在 message 中说明错误原因）或 `result: Timeout`。设计中不进行多次重试，因此出现错误即结束本次处理。

gRPC 调用代码示例

以下为 Operator 中使用 Go 调用 unit-agent gRPC 接口的示例代码片段：

```
go
1  import (
2      "context"
3      "time"
4      "fmt"
5      "google.golang.org/grpc"
6      "google.golang.org/grpc/credentials/insecure"
7      pb "example.com/unitagent"
8  )
9
10 // 调用 unit-agent 的 Backup 方法
11 func callBackup(podIP string, params map[string]string) error {
12     // 建立与 unit-agent 的 gRPC 连接（无加密）
13     conn, err := grpc.Dial(fmt.Sprintf("%s:9090", podIP),
14         grpc.WithTransportCredentials(insecure.NewCredentials()))
15     if err != nil {
16         return fmt.Errorf("无法连接 unit-agent: %v", err)
17     }
18     defer conn.Close()
19     client := pb.NewUnitAgentClient(conn)
20 }
```

```

21 // 设置超时时间
22 ctx, cancel := context.WithTimeout(context.Background(), 10*time.Minute)
23 defer cancel()
24
25 // 构造请求参数
26 req := &pb.BackupRequest{
27     Mode:    params["mode"],
28     Bucket:  params["bucket"],
29     // 其他参数...
30 }
31 // 发起 gRPC 调用
32 res, err := client.Backup(ctx, req)
33 if err != nil {
34     return fmt.Errorf("执行 Backup 失败: %v", err)
35 }
36 // 处理返回结果 (示例直接打印)
37 fmt.Printf("Backup 结果: %s\n", res.Message)
38 return nil
39 }

```

在上述示例中，使用

`grpc.WithTransportCredentials(insecure.NewCredentials())` 构造非加密连接，调用 `Backup` 方法，并处理返回值。类似地，可以调用其他操作（`Restore`、`ReloadConfig` 等），只需替换请求类型和方法名即可。

安全性建议

- **gRPC 认证与加密**：默认示例中使用了无加密的连接（`insecure.NewCredentials()`）。在生产环境中，建议在 operator 与 unit-agent 间使用 TLS 加密，或者采取其他认证机制（如在元数据中传递短期凭证）。例如，可以为 unit-agent 配置 mTLS 证书，Operator 使用相应的客户端证书访问，以确保身份校验和数据加密。若无法使用 TLS，也应确保 Operator 运行在可信网络环境中，避免证书/Token 泄露。
- **资源隔离**：建议将 Unit、unit-agent 和 GrpcCall Operator 部署在同一命名空间（或受控网络区域）内，并使用标签或命名约定加以区分。这样可以结合 Kubernetes 命名空间和网段规则进行更加细致的访问控制。

总结

GrpcCall Operator 通过定义新的 GrpcCall CRD，使得调用 unit-agent 的操作变得更加 Kubernetes 化和可观测。每个 GrpcCall 实例代表一次 gRPC 调用操作，Operator 监听 CRD 并执行对应调用，结果记录在 CR 的状态中。这种模式免除了手动创建 Job 的繁琐，同时利用了 Operator 模式的设计原则，将“备份、还原等运维操作”自动化、以声明式资源的形式提交。在设计过程中，我们参考了 Kubernetes Operator 模式的最佳实践：Operator 是管理自定义资源的控制器kubernetes.io，常见用

例包括应用部署和备份恢复等kubernetes.io。结合安全建议（最小权限、网络策略等），GrpcCall Operator 能在保证安全性的前提下有效简化操作流程。